



Security Assessment Draft Report



Silo Enterprise Continuous Audit

March 2026

Prepared for Silo

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	4
Findings Summary.....	5
Severity Matrix.....	5
Detailed Findings.....	6
Medium Severity Issues.....	8
M-01: `onDebtTransfer` fails to update stale `borrowerCollateralSilo` causing wrong solvency configuration for debt recipients in two-way markets.....	8
M-02: `_updateSalt` incorrectly assumes the salt parameter is physically at the end of the calldata, silently corrupting dynamic oracle configurations.....	10
Low Severity Issues.....	11
L-01: Defaulting-hook ownership transfers do not migrate whitelist admin, allowing former owners to retain liquidation censorship.....	11
L-02: One-sided oracle validation accepts non-quote assets as price = 1, enabling catastrophic market mispricing.....	12
L-03: Fractional interest rounding divergence in view functions causes `closeLeveragePosition` to revert and breaks `repayAll`.....	13
L-04: UniswapV3Oracle TWAP period can silently collapse to a manipulable near-spot-price window..	15
Informational Severity Issues.....	16
I-01: Hardcoded absolute DEFAULT_LOST_THRESHOLD enables significant fund drain for low-decimal tokens.....	16
I-02: Curator Can Bypass Timelock and Inject Malicious Claiming Logics via Trusted Factory.....	17
I-03: Defaulting liquidations front-running in bad debt case.....	18
I-04: State-changing ERC4626 calls trust market-reported results, enabling silent asset loss on misbehaving markets.....	19
Disclaimer.....	20
About Certora.....	20

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Silo Enterprise Continuous Audit	https://github.com/silo-finance/silo-contracts-v3	9ff762fad10eb1a20081efc1652a7f0b8a1d0367	Solidity

Project Overview

This document describes the security review of **Silo Enterprise Continuous Audit**. The work was undertaken in **March 2026**.

The following contract list is included in our scope:

silo-core/contracts/**

silo-oracles/contracts/**

silo-vaults/contracts/**

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Silo Finance is a decentralized lending protocol designed to reduce systemic risk in DeFi by isolating each asset in its own lending market. Instead of sharing collateral pools across many assets (like traditional lending protocols), every asset has a separate “silo” market, meaning risk from one asset (e.g., a bad debt event) cannot spread to others. Users can supply assets to earn yield or borrow against collateral within that asset’s silo while maintaining strict risk isolation.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	2	-	-
Low	4	-	-
Informational	4	-	-
Total	10	-	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	<code>`onDebtTransfer`</code> fails to update stale <code>`borrowerCollateralSilo`</code> causing wrong solvency configuration for debt recipients in two-way markets	Medium	Pending
M-02	<code>_updateSalt</code> incorrectly assumes the salt parameter is physically at the end of the calldata, silently corrupting dynamic oracle configurations	Medium	Pending
L-01	Defaulting-hook ownership transfers do not migrate whitelist admin, allowing former owners to retain liquidation censorship	Low	Pending
L-02	One-sided oracle validation accepts non-quote assets as price = 1, enabling catastrophic market mispricing	Low	Pending
L-03	Fractional interest rounding divergence in view functions causes <code>`closeLeveragePosition`</code> to revert and breaks <code>`repayAll`</code>	Low	Pending
L-04	UniswapV3Oracle TWAP period can silently collapse to a manipulable near-spot-price window	Low	Pending

I-01	Hardcoded absolute DEFAULT_LOST_THRESHOLD enables significant fund drain for low-decimal tokens	Informational	Pending
I-02	Curator Can Bypass Timelock and Inject Malicious Claiming Logics via Trusted Factory	Informational	Pending
I-03	Defaulting liquidations front-running in bad debt case	Informational	Pending
I-04	State-changing ERC4626 calls trust market-reported results, enabling silent asset loss on misbehaving markets	Informational	Pending

DRAFT

Medium Severity Issues

M-01: `onDebtTransfer` fails to update stale `borrowerCollateralSilo` causing wrong solvency configuration for debt recipients in two-way markets

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: SiloConfig.sol	Status: Pending	

Description:

The `borrowerCollateralSilo` mapping in `SiloConfig` is never cleared after a borrower fully repays their position. No repayment flow—whether through `repay()`, `SiloLendingLib.repay()`, or any other debt-settlement path—resets `borrowerCollateralSilo[borrower]` back to `address(0)`. This stale state combined dangerously with `onDebtTransfer()`, which only updates the mapping when `borrowerCollateralSilo[_recipient] == address(0)`, creating a concrete path to bypass the protocol's same-asset borrowing restriction.

A valid exploit flow is:

1. Alice borrows from `SILO0`, which sets `borrowerCollateralSilo[Alice] = SILO1`.
2. Alice fully repays her debt in `SILO0`. However, `borrowerCollateralSilo[Alice]` remains unchanged and still points to `SILO1`.
3. Bob, who has outstanding debt shares in `SILO1`, transfers those shares to Alice.
4. During `onDebtTransfer()`, the protocol checks whether Alice has debt in the other silo. Because Alice has already repaid and holds no live debt in `SILO0`, this check passes.
5. However, the protocol does **not** refresh Alice's collateral silo assignment, because the mapping is already non-zero. As a result, Alice now holds `SILO1` debt while `borrowerCollateralSilo[Alice]` still equals `SILO1`.

This is the critical state corruption: when `getConfigsForSolvency()` later resolves Alice's account, both `collateralConfig` and `debtConfig` resolve to `SILO1`. In other words, the protocol treats Alice's position as if the same asset can back the same asset debt. Because the collateral token and

debt token are identical, `SiloSolvencyLib` falls into its same-asset assumption and values the position at a 1:1 rate without relying on an oracle. That effectively enables Alice's `SIL01` deposits to collateralize her `SIL01` borrow.

This behavior directly defeats the protocol's intended same-asset borrowing prohibition. `borrowSameAsset()` is supposed to be unreachable and explicitly reverts with `Deprecated()`, yet the stale `borrowerCollateralSilo` state allows an attacker to reconstruct the forbidden configuration indirectly through debt-share transfers. The impact is especially severe in one-way markets, where same-asset borrowing must remain impossible by design.

Recommendations:

Update `onDebtTransfer` to consistently apply the correct collateral silo for the recipient based on the debt silo being transferred, rather than conditionally setting it only if it is zero.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

M-02: `_updateSalt` incorrectly assumes the salt parameter is physically at the end of the calldata, silently corrupting dynamic oracle configurations

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: SiloDeployer.sol	Status: Pending	

Description:

In `SiloDeployer.sol`, the `_createOracle` function attempts to dynamically inject a unique, deterministic salt into the provided oracle factory calldata by overwriting the last 32 bytes using inline assembly (`mstore(pointer, salt)`). This strictly assumes the `_externalSalt` parameter is physically at the very end of the ABI-encoded calldata. However, whenever preceding parameters contain dynamic types (e.g., strings in DIA configurations or bytes in ManageableOracle configurations), the **dynamic string contents** are encoded in the tail section. When `_updateSalt` executes, it blindly overwrites the string character data in the tail instead of the `_externalSalt` head. This can silently corrupt string parameters like DIA feed keys, rendering the deployed oracle unusable.

Recommendations:

Consider not attempting to hot-patch ABI-encoded calldata via offset assumptions when the layout is susceptible to dynamic tail data. Enforce a standardized interface for oracle factories and provide the opaque encoded configuration alongside the salt as explicitly distinct arguments.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

Low Severity Issues

L-01: Defaulting-hook ownership transfers do not migrate whitelist admin, allowing former owners to retain liquidation censorship

Severity: Low	Impact: Low	Likelihood: Low
Files: Whitelist.sol GaugeHookReceiver.sol	Status: Pending	

Description:

Defaulting liquidations are guarded by `onlyAllowedOrPublic`, which relies on the Whitelist admin. The `DEFAULT_ADMIN_ROLE` for the whitelist is granted to the initial owner during deployment. However, hook ownership is managed separately through `Ownable1and2Steps`, and subsequent ownership transfers do not migrate or revoke the `DEFAULT_ADMIN_ROLE`. If ownership is transferred post-deployment, the previous owner retains the ability to grant or revoke the `ALLOWED_ROLE`, allowing them to censor or monopolize defaulting liquidations independently of the new owner. In SiloHookV3 markets, where defaulting is the only permitted liquidation path, this effectively allows a former admin to block liquidations entirely.

Recommendations:

Consider synchronizing whitelist administration with ownership changes. Override ownership transfer/acceptance so the new owner is granted `DEFAULT_ADMIN_ROLE` and the old owner is revoked in the same flow, or derive whitelist management directly from owner authority rather than a separate AccessControl domain.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

L-02: One-sided oracle validation accepts non-quote assets as price = 1, enabling catastrophic market mispricing

Severity: Low	Impact: Low	Likelihood: Low
Files: Views.sol	Status: Pending	

Description:

`Views.validateSiloInitData()` only verifies that configured oracles share the same `quoteToken`, but skips validation entirely if an oracle is omitted. It does not verify that the asset lacking an oracle is actually the quote token itself. Later, `SiloSolvencyLib.getPositionValues()` assumes that any asset with an `address(O)` oracle has a price of 1 quote unit per raw asset unit. A trusted deployer or misconfiguration could create a market where Token A has an oracle quoted in Token Q, but Token B lacks an oracle and is *not* Token Q. In this scenario, Token B is implicitly mispriced 1:1 with raw asset units. If token B has different decimals or a vastly different value than the quote asset, collateral will be massively overvalued or debt massively undervalued.

Recommendations:

Consider hardening market validation so that an omitted oracle is only allowed when the corresponding asset is provably the shared quote asset. Reject one-sided oracle setups otherwise.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

L-03: Fractional interest rounding divergence in view functions causes `closeLeveragePosition` to revert and breaks `repayAll`

Severity: Low	Impact: Low	Likelihood: Low
Files: Views.sol LeverageUsingSiloFlashloan.sol SiloRouterV2Implementation.sol	Status: Pending	

Description:

View functions like `maxRepay` fail to account for fractional interest rollover, while state-changing `repay` paths apply it. This can cause the estimated repayment amount to underestimate the true debt by 1 wei. When a user closes a leverage position `closeLeveragePosition`, the exact flashloaned amount might be 1 wei short, causing a revert:

```
silowithDebt.repayShares(_getBorrowerTotalShareDebtBalance(silowithDebt),
_txMsgSender);
```

Additionally, `repayAll` can leave residual dust debt:

```
function repayAll(ISilo _silo) external payable virtual returns (uint256 shares)
{
    uint256 repayAmount = _silo.maxRepay(msg.sender);
    IERC20 asset = IERC20(_silo.asset());

    transferFrom(asset, address(this), repayAmount);
    approve(asset, address(_silo), repayAmount);

    shares = repay(_silo, repayAmount);
}
```

Recommendations:

Due to the low impact, consider acknowledging this issue.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

DRAFT

L-04: UniswapV3Oracle TWAP period can silently collapse to a manipulable near-spot-price window

Severity: Low	Impact: Low	Likelihood: Low
Files: UniswapV3OracleFactory.sol	Status: Pending	

Description:

When the Uniswap V3 pool's observation buffer does not cover the requested TWAP period, the `observe()` call reverts. The oracle's `_calculatePeriodAndTicks` catches this and dynamically shrinks the TWAP period to exactly match the oldest available timestamp (`block.timestamp - oldestObservationTimestamp`). Because the factory `create()` does not enforce pool buffer cardinality checks (an explicitly documented limitation), deploying against a newly initialized or low-cardinality pool shrinks the TWAP to a near-spot price. This makes the oracle more manipulable via flash-loan sandwich attacks during the initial buffer fill period.

Recommendations:

Consider enforcing a strict minimum acceptable bound on the adjusted period during the fallback to ensure adequate TWAP protection.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

Informational Severity Issues

I-01: Hardcoded absolute DEFAULT_LOST_THRESHOLD enables significant fund drain for low-decimal tokens

Files:

[SiloVault.sol](#)

Description:

The `_priceManipulationCheck` function uses `DEFAULT_LOST_THRESHOLD = 1e6` to determine the maximum acceptable loss when depositing to an underlying market. Because this threshold is a raw asset-unit constant, its economic meaning varies wildly with token decimals. For an 8-decimal token like WBTC, `1e6` represents exactly 0.01 WBTC, which carries a massive dollar value. An attacker could potentially manipulate the target market's share price or exploit rounding so each deposit loses just under the threshold, silently harvesting that value from vault depositors during every operation. While the guardian can set per-market thresholds to mitigate this, the default puts low-decimal assets at significant risk before custom overrides are applied.

Recommendations:

Instead of a hardcoded absolute value, use a percentage-based threshold (e.g., a fraction of the deposited `_assets` amount), or adjust the default threshold dynamically based on the underlying token's decimals. Alternatively, require the deployer to explicitly configure an appropriate threshold for the token during market initialization.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

I-02: Curator Can Bypass Timelock and Inject Malicious Claiming Logics via Trusted Factory

Files:

[VaultIncentivesModule.sol](#)

Description:

When a claiming logic is deployed via a trusted factory, a curator can submit it using a fast-path that bypasses the timelock entirely (`timelock = 0`). Because the trusted factory only verifies the origin of the logic and not its instance parameters (such as the underlying controller address), a compromised curator could deploy a malicious logic pointing to an always-reverting controller and instantly inject it into the vault. This allows the curator to immediately DoS the vault's `_claimRewards()` process without any timelock delay, temporarily bricking the vault until the owner intervenes.

Recommendations:

Consider not treating factory provenance alone as sufficient trust. Require accepted logics to be parameter-validated on-chain against the vault and market. Alternatively, enforce a minimum non-zero timelock even for trusted factory logics, or require owner approval for acceptance.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

I-03: Defaulting liquidations front-running in bad debt case

Files:

[PartialLiquidationByDefaulting.sol](#)

Description:

The liquidation by defaulting decreases the `totalAssets` of the `debtSilo` collateral share token. In the bad debt scenario, it means that lenders are expected to lose value as a result of the defaulting liquidation. Active lenders may perform the following sequence of actions to shift the bad debt to other lenders

- 1) Execute a normal liquidation call, to seize the profit.
- 2) Withdraw collateral
- 3) Execute a defaulting liquidation to socialize the bad debt among the other lenders.

Even though this attack allows skipping bad debt socialization, due to the defaulting liquidation logic, accruing bad debt is unlikely. Furthermore, this race condition requires front-running the liquidation keepers and a big oracle shift that will instantly cause liquidations.

Recommendations:

Consider acknowledging and documenting this issue as this is a part of the liquidation by defaulting design. Alternative approaches may include forcing a withdrawal delay for lenders or others, which may require refactoring and code changes.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

I-04: State-changing ERC4626 calls trust market-reported results, enabling silent asset loss on misbehaving markets

Files:

[SiloVault.sol](#)

Description:

The vault performs critical state-changing market interactions in the supply path using external ERC4626 return values as the source of truth, without verifying the vault's real underlying balance deltas. In `_marketSupply`, a market supply is treated as successful if `deposit()` returns a non-zero `gotShares`. Unlike the withdrawal paths (which use `_checkAfterWithdraw`), the supply path never verifies that the asset outflow corresponded to a real position credited to the vault. A compromised market can pull assets, return an arbitrary share amount, and consistently lie via `previewRedeem()`, leading to bounded asset loss.

Recommendations:

Do not trust ERC4626 return values alone. For the supply-side market interactions, snapshot the vault's underlying asset balance before/after and the vault's market share balance before/after. Require actual share receipt by the vault before considering the deposit successful, reverting on any inconsistencies.

Customer Response:

Awaiting a response

Fix Review:

Awaiting a response

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.